
CAPSTONE DEFENSE REFERENCE

Academic Evaluation System

Frontend and Backend Architecture — how the system works, how the code is organized, and what each major layer does.

User → React Interface → Laravel API → Business Rules → MySQL Database → JSON Response → Updated Interface

Technology: React 19, React Router, Axios, Laravel 12, PHP 8.2+, JWT, Eloquent, MySQL

Prepared: September 2026

1. Whole-System Overview

Short defense explanation

“Our Academic Evaluation System uses a client-server architecture. The React frontend is responsible for the user interface, role-based portals, forms, reports, and interaction. It communicates with a Laravel REST API through Axios. The backend authenticates the user, validates requests, applies academic and permission rules, reads or updates the MySQL database, and returns JSON. React stores the returned data in component state and automatically updates the page.”



Frontend responsibilities

- Display pages appropriate to the user’s role and permissions.
- Collect input and provide immediate form feedback.
- Send authenticated API requests and display responses.
- Handle responsive layout, accessibility, reports, and exports.

Backend responsibilities

- Authenticate JWTs and enforce authoritative access rules.
- Validate requests and academic operations.
- Persist curricula, evaluations, users, and related records.
- Provide auditing, imports, analytics, and database backups.

Important defense point: The frontend may hide a page or button, but that is only a usability control. Real security must be enforced by backend middleware and controller permission checks.

2. Example: Student Evaluation Flow

1. A permitted staff member opens the shared student evaluation workspace.
2. React requests the student directory and the selected student’s academic record.
3. Axios attaches the saved JWT as a Bearer token.
4. Laravel verifies the JWT and evaluation access before running the controller.
5. The backend loads curriculum, grades, prerequisites, electives, standing, and transfer credits.
6. The evaluator edits grades or plans in React state and confirms the action.
7. Laravel validates and saves the permitted operation, then returns JSON.
8. The frontend reloads the record and shows the updated evaluation or report.

3. Frontend Architecture

- React 19
- React Router 6
- Axios
- CRA 5 + CRACO
- SweetAlert2
- jsPDF

Frontend defense script

“The frontend is a React single-page application. `index.js` starts React, and `App.js` initializes authentication, the system guide, session controls, network feedback, and routing. `AuthContext` stores the current user and permissions. React Router opens the correct portal, while reusable components render modules such as curriculum management and student evaluation. Components call the Laravel API through a centralized Axios client, then update React state using the returned JSON.”

Code area	Function	Important file
Entry point	Mounts React and imports global styles.	<code>src/index.js</code>
Application shell	Defines providers, global UI, lazy loading, and browser routes.	<code>src/App.js</code>
Authentication	Restores the user, logs in/out, provides role and permission helpers, and stores session policy.	<code>src/context/AuthContext.js</code>
API communication	Sets the API URL, adds JWT headers, refreshes expired tokens, and reports network failures.	<code>src/api/axios.js</code>
Role portals	Show permission-controlled navigation and feature modules.	<code>src/components/{role}/</code>
Shared features	Reusable sidebar, evaluation view, system guide, selects, modals, and pagination.	<code>src/components/common/</code>
Utilities	PDF generation, validation, SweetAlert helpers, and network recovery.	<code>src/utils/</code>

Components

Reusable UI or feature functions that return JSX.

Props

Inputs passed from a parent component to a child.

State

Changing data such as forms, filters, records, and active tabs.

Hooks

Manage state, effects, memoized values, callbacks, focus, and sessions.

Context

Shares authenticated user and guide state throughout the component tree.

Router

Maps URLs to public pages or protected role portals.

4. Frontend Portals and Shared Behavior

Portal	Purpose	Route
Public	Landing, login, guest curriculum, and credit simulation.	/, /login, /guest
Admin	Users, roles, lookup data, curriculum, security, imports, audits, and backups.	/admin
Student	Dashboard, profile, and curriculum progress.	/student
Dean	Department records, evaluation, and decision analytics.	/dean
Adviser/Evaluator	Classes, grades, student evaluation, and adviser analytics.	/evaluator
Program Head	Program-scoped evaluation, curriculum, and analytics.	/program-head
Secretary	Permission-controlled student and curriculum administration.	/secretary

State and storage

- React state: live page, form, filter, modal, and API data.
- Context: authenticated user and guide state.
- `localStorage` : JWT, tab/sidebar preferences, guide status, and some evaluation plans.
- `sessionStorage` : recoverable non-password form drafts.

Accessibility and responsiveness

- Skip link and semantic main content.
- Visible keyboard focus and reduced-motion support.
- Focus trapping and restoration for shared dialogs.
- Keyboard-accessible tabs and searchable selects.
- Responsive layouts and sidebar behavior.

Verified frontend limitations:

- There are currently no frontend test files, and CRACO removes ESLint from webpack compilation.
- The JWT is stored in `localStorage` , making strong XSS prevention essential.
- `/student` requires login but does not explicitly require the Student role.
- Unknown URLs have no catch-all 404 route.
- Evaluation row saves use separate concurrent requests, so partial persistence is possible.
- `StudentEvaluationView.js` is over 8,000 lines and combines many responsibilities.
- html2canvas-based PDFs are rasterized and are less searchable and accessible.

5. Backend Architecture

Laravel 12

PHP 8.2+

JWT Auth

Eloquent ORM

MySQL

Backend defense script

"The backend is a Laravel REST API. Requests enter through `routes/api.php` . Middleware checks the JWT, unsafe input, and evaluation access. The matching domain controller validates the request and coordinates models or services. Services contain complex rules such as authentication security, academic standing, prerequisites, promotion, imports, transfer credits, and backups. Eloquent models represent database tables and relationships. Laravel then returns JSON to the frontend, while audit observers record important model changes."



Layer	Function	Location
Bootstrap	Registers routes, global middleware, aliases, and API exception behavior.	<code>bootstrap/app.php</code>
Routes	Map HTTP methods and URLs to controllers and middleware.	<code>routes/api.php</code>
Middleware	Authenticate, reject unsafe input, add security headers, and enforce evaluation access.	<code>app/Http/Middleware/</code>
Controllers	Validate requests, authorize operations, call services/models, and return responses.	<code>app/Http/Controllers/</code>
Services	Hold reusable and complex business rules.	<code>app/Services/</code>
Models	Map database tables, keys, casts, and relationships.	<code>app/Models/</code>
Migrations	Version changes to the database schema.	<code>database/migrations/</code>

6. Backend Business Domains

Domain	Main responsibility
Auth	Login, logout, token refresh, registration, current user, and password changes.
Identity	User accounts, roles, permissions, and per-user access overrides.
Profiles	Student, dean, and faculty/adviser profile-specific data.
Curriculum	Programs, subjects, curriculum versions, requisites, lookup data, and elective slots.
Evaluation	Grades, academic standing, promotion, subject placement, reports, and analytics.
Transfer Credit	Prior schools, external subjects, equivalencies, simulations, and credited subjects.
Administration	Security settings, audit logs, CSV imports, and database backups.

7. Core Data Relationships

Identity

`TblUser` → `Role` → `Permissions`

A user can also have a student, dean, or faculty profile.

Organization and curriculum

`Campus` → `Department` → `Program` →

`CurriculumHeader` → `Curriculum`

Curriculum rows point to a subject or an elective slot and include year and semester.

Academic record

`StudentProfile` → `Evaluation` → `Subject`

Evaluations also reference term, section, modality, grade components, and evaluator.

Transfer credit

`Student` → `CreditEvaluation` →

`CreditEvaluationDetail`

External school subjects are mapped to local subjects through equivalencies.

8. Security and Reliability

- `auth:api` uses JWT and the `TblUser` provider.
- `SecurityHeaders` adds browser security policies to responses.
- `RejectUnsafeInput` checks API input and normalizes contact numbers.
- `EvaluationAccessMiddleware` checks evaluation privileges and restricts deletion.
- Laravel request validation rejects invalid payloads with HTTP 422.
- Transactions protect many multi-table imports, evaluations, promotions, and credit operations.
- `AuditModelObserver` records important model changes.

9. Database, Operations, and Verified Limitations

Database lifecycle

- 81 migrations record incremental schema changes.
- 11 seeders provide permissions, users, subjects, electives, and demonstration records.
- Most models map legacy `tbl_*` names and custom primary keys.

Operational tools

- `scripts/imports/` : one-off curriculum imports.
- `scripts/maintenance/` : repair and cleanup utilities.
- `storage/app/backups/` : generated database backups.
- Scheduled commands handle INC expiry and backup checks.

Verified backend limitations:

- `POST /api/jwt/register` is public and accepts a caller-selected existing `role_id`. This can allow privileged self-registration unless restricted before deployment.
- Comments saying “admin only” are not enforcement. Many permissions are checked inside controllers rather than attached directly to route groups.
- Fresh migrations do not create several foundational legacy tables, including users, students, subjects, programs, and campuses.
- The setup instructions reference `capstone_db.sql`, but that file is absent. An empty database therefore cannot currently be rebuilt reliably from the repository alone.
- Backend automated coverage contains only two unit-test suites and does not exercise controllers, MySQL schema, imports, or backups.

10. Common Defense Questions

Question	Brief answer
Why separate frontend and backend?	It separates presentation from business rules and data access, allowing each side to be developed, tested, and secured independently.
Why use JWT?	It lets the stateless API identify authenticated users through a signed Bearer token on each protected request.
Why use services?	Services isolate complex reusable rules and prevent large controllers from duplicating domain logic.
Why use migrations?	They provide an ordered, version-controlled history of database changes. In this project they currently supplement, rather than fully replace, the legacy base schema.
How are roles controlled?	The backend provides role and permission data; the frontend filters modules, while the backend must authorize sensitive API operations.
How is data consistency protected?	Laravel validation, foreign keys, transactions, controller checks, and audit logging protect many operations. Known non-atomic frontend save flows remain improvement targets.

This guide describes the verified repository structure as of September 2026. Identified limitations are included so the architecture can be defended accurately rather than overstated.